



Universidad Nacional Autónoma de México



Facultad de Ingeniería

Práctica 3:

Circuitos de 1 y 2 orden.

Integrantes:

Espinoza Matamoros Percival Ulises - 320025561

Flores Colin Victor Jaziel - 320266083

García Cortés Adolfo de Jesús - 320317264

Lara Hernandez Angel Husiel - 320060829

Lugo Manzano Rodrigo - 320206968

Microcomputadoras

Grupo: 03 - Semestre: 2026-2

Calf: _____



1. Objetivo:

2. Desarrollo:

Los parámetros del sistema son:

- **Frecuencia de muestreo:** $f_s = 8000$ Hz
- **Período de muestreo:** $T = \frac{1}{f_s} = \frac{1}{8000} = 1.25 \times 10^{-4}$ s = 125 μ s
- **Frecuencia de Nyquist:** $f_N = \frac{f_s}{2} = 4000$ Hz
- **Entrada analógica:** GP26 (ADC0)
- **Generación de señal de prueba:** GP3 (PWM, onda cuadrada)

Para el uso de los hilos en la pico: el **Núcleo 1** ejecuta la ecuación en diferencias del filtro activo con temporización precisa a $f_s = 8000$ Hz, mientras que el **Núcleo 0** procesa todas las entradas que se van recibiendo.

2.1. Filtro Pasa Bajas de Primer Orden (IIR Butterworth)

Se selecciona una frecuencia de corte $f_c = 500$ Hz para el filtro pasa bajas. Esto por:

1. **Criterio de Nyquist:** La frecuencia de corte debe satisfacer $f_c < f_N = 4000$ Hz, con $f_c = 500$ Hz.
2. **Relación f_c/f_s :** La relación $f_c/f_s = 500/8000 = 0.0625$ se encuentra en el rango de $(0.01 < f_c/f_s < 0.45)$, lo que hace que la transformada bilineal no introduzca distorsión por efecto de frequency warping.

Circuito eléctrico

El filtro pasa bajas analógico de primer orden se implementa con un circuito RC serie, donde la salida se toma sobre el capacitor:

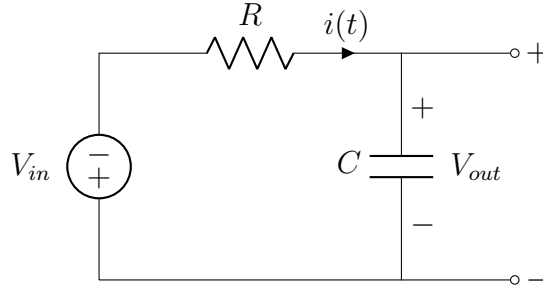


Figura 1: Circuito RC pasa bajas de primer orden.

calculo de componentes:

La frecuencia de corte del circuito RC está dada por:

$$f_c = \frac{1}{2\pi RC} \implies \omega_c = \frac{1}{RC} = 2\pi f_c \quad (1)$$

Seleccionando $C = 100$ nF (serie E24), se calcula la resistencia necesaria para obtener $f_c = 500$ Hz:

$$R = \frac{1}{2\pi f_c C} = \frac{1}{2\pi(500)(100 \times 10^{-9})} = \frac{1}{3.14159 \times 10^{-4}} = 3183.1 \, \Omega \approx 3.18 \, \text{k}\Omega \quad (2)$$

Se utiliza una resistencia de $R = 3.3 \, \text{k}\Omega$ (serie E24), lo cual resulta en una frecuencia de corte real de:

$$f_{c,\text{real}} = \frac{1}{2\pi(3300)(100 \times 10^{-9})} = \frac{1}{2.0735 \times 10^{-3}} = 482.3 \, \text{Hz} \quad (3)$$

Esta desviación de $\approx 3.5 \%$ respecto al valor teórico es aceptable.

2.1.1. Obtención de la función de transferencia $H(s)$

Aplicando la ley de Kirchhoff al circuito RC en el dominio de Laplace:

$$V_{in}(s) = R \cdot I(s) + \frac{1}{sC} \cdot I(s) = I(s) \left(R + \frac{1}{sC} \right) \quad (4)$$

El voltaje de salida sobre el capacitor es:

$$V_{out}(s) = \frac{1}{sC} \cdot I(s) \quad (5)$$



Despejando la corriente de la primera ecuación y sustituyendo:

$$H(s) = \frac{V_{out}(s)}{V_{in}(s)} = \frac{\frac{1}{sC}}{R + \frac{1}{sC}} = \frac{1}{sRC + 1} \quad (6)$$

Multiplicando numerador y denominador por $\omega_c = 1/(RC)$:

$$H(s) = \frac{\omega_c}{s + \omega_c}, \quad \omega_c = 2\pi(500) = 3141.59 \text{ rad/s} \quad (7)$$

Esta es la función de transferencia de un filtro Butterworth pasa bajas de primer orden, con ganancia unitaria en DC ($H(0) = 1$) y atenuación de -3 dB en $\omega = \omega_c$.

2.1.2. Discretización mediante la transformada bilineal

Paso 1: Pre-distorsión de frecuencia. La transformada bilineal introduce una distorsión no lineal entre las frecuencias analógica y digital. Para garantizar que la frecuencia de corte sea exactamente la misma y se mantenga, se calcula la frecuencia analógica pre-distorsionada:

$$\Omega_a = \tan\left(\frac{\pi f_c}{f_s}\right) = \tan\left(\frac{\pi \cdot 500}{8000}\right) = \tan\left(\frac{\pi}{16}\right) \quad (8)$$

Evaluyendo:

$$\Omega_a = \tan(0.19635 \text{ rad}) = 0.19891 \quad (9)$$

$$H(s) = \frac{\Omega_a}{s + \Omega_a} \quad (10)$$

Paso 2: Sustitución de la variable bilineal.

$$s = \frac{z - 1}{z + 1} \quad (11)$$

Sustituyendo en $H(s)$:

$$H(z) = \frac{\Omega_a}{\frac{z - 1}{z + 1} + \Omega_a} \quad (12)$$

simplificando:



Multiplicando numerador y denominador por $(z + 1)$:

$$H(z) = \frac{\Omega_a(z + 1)}{(z - 1) + \Omega_a(z + 1)} \quad (13)$$

Expandiendo el denominador:

$$\begin{aligned} (z - 1) + \Omega_a(z + 1) &= z - 1 + \Omega_a z + \Omega_a \\ &= z(1 + \Omega_a) + (\Omega_a - 1) \end{aligned} \quad (14)$$

Dividiendo numerador y denominador entre $(1 + \Omega_a)$:

$$H(z) = \frac{\frac{\Omega_a}{1 + \Omega_a} (z + 1)}{z + \frac{\Omega_a - 1}{1 + \Omega_a}} = \frac{\frac{\Omega_a}{1 + \Omega_a} (z + 1)}{z - \frac{1 - \Omega_a}{1 + \Omega_a}} \quad (15)$$

Paso 3: Identificación con la forma estándar. Comparando con la forma canónica

$$H(z) = \frac{A_0 z + A_1}{z - B_1}:$$

$$A_0 = A_1 = \frac{\Omega_a}{1 + \Omega_a} = \frac{0.19891}{1 + 0.19891} = \frac{0.19891}{1.19891} = 0.16591 \quad (16)$$

$$B_1 = \frac{1 - \Omega_a}{1 + \Omega_a} = \frac{1 - 0.19891}{1 + 0.19891} = \frac{0.80109}{1.19891} = 0.66818 \quad (17)$$

Por lo tanto, la función de transferencia discreta es:

$$H(z) = \frac{0.16591 (z + 1)}{z - 0.66818} = \frac{0.16591 z + 0.16591}{z - 0.66818} \quad (18)$$

2.1.3. Ecuación en diferencias

A partir de $H(z) = Y(z)/U(z)$:

$$Y(z) (z - B_1) = U(z) (A_0 z + A_1) \quad (19)$$

Dividiendo ambos lados entre z :

$$Y(z) - B_1 z^{-1} Y(z) = A_0 U(z) + A_1 z^{-1} U(z) \quad (20)$$

Aplicando la transformada z inversa (donde z^{-1} representa un retardo de una muestra):

$$y(k) = A_0 \cdot u(k) + A_1 \cdot u(k-1) + B_1 \cdot y(k-1) \quad (21)$$

Sustituyendo los valores numéricos:

$$y(k) = 0.16591 \cdot u(k) + 0.16591 \cdot u(k-1) + 0.66818 \cdot y(k-1) \quad (22)$$

Coefficientes:

Tabla 1: Coeficientes del filtro pasa bajas de primer orden ($f_c = 500$ Hz, $f_s = 8000$ Hz).

Coeficiente	Expresión	Valor numérico
A_0	$\frac{\Omega_a}{1 + \Omega_a}$	0.16591
A_1	$\frac{\Omega_a}{1 + \Omega_a}$	0.16591
A_2	—	0.0
B_1	$\frac{1 - \Omega_a}{1 + \Omega_a}$	0.66818
B_2	—	0.0

Verificación en DC ($z = 1$):

$$H(1) = \frac{A_0 + A_1}{1 - B_1} = \frac{0.16591 + 0.16591}{1 - 0.66818} = \frac{0.33182}{0.33182} = 1.0 \quad (23)$$

De este modo, se verifica que la componente DC no sufre atenuación.

2.1.4. Diagrama de flujo de señal

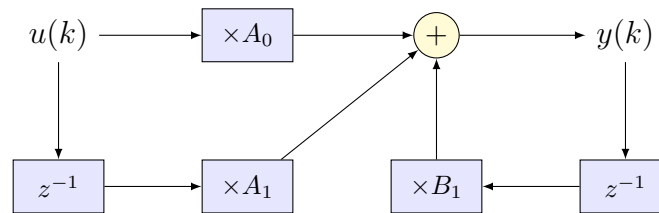


Figura 2: Diagrama de flujo de señal del filtro pasa bajas de primer orden (Forma Directa I).



2.2. Diseño del Filtro Pasa Altas de Segundo Orden (Circuito RLC)

Se selecciona una frecuencia de corte $f_c = 800$ Hz para el filtro pasa altas de segundo orden. Para esto, se toma en cuenta lo siguiente:

1. **Criterio de Nyquist:** Se cumple $f_c = 800$ Hz $< f_N = 4000$ Hz.
2. **Relación:** La relación $f_c/f_s = 800/8000 = 0.1$ se ubica en la zona central del rango recomendado, donde la transformada bilineal presenta mínima distorsión.

Circuito:

El filtro pasa altas de segundo orden se implementa obligatoriamente con un circuito **RLC serie**, ya que para obtener una función de transferencia de segundo grado se requieren al menos dos elementos almacenadores (capacitor e inductor).

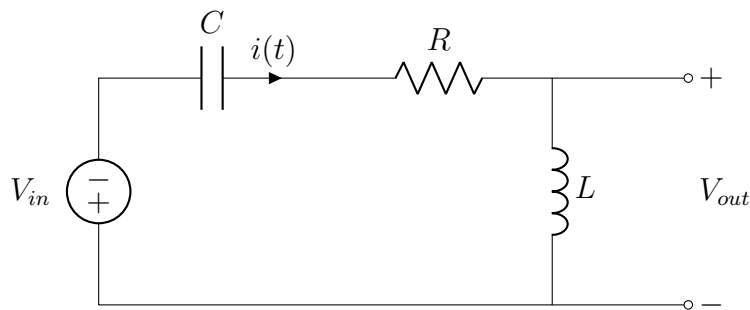


Figura 3: Circuito RLC serie pasa altas de segundo orden (salida sobre el inductor L).

calculo de los componentes:

Del circuito RLC serie se obtiene:

$$H(s) = \frac{s^2}{s^2 + \frac{R}{L}s + \frac{1}{LC}} \quad (24)$$

Comparando con la forma canónica Butterworth de segundo orden:

$$H(s) = \frac{s^2}{s^2 + \sqrt{2}\omega_c s + \omega_c^2} \quad (25)$$



Se identifican las relaciones de diseño:

$$\frac{1}{LC} = \omega_c^2, \quad \frac{R}{L} = \sqrt{2}\omega_c \quad (26)$$

Con $\omega_c = 2\pi(800) = 5026.55$ rad/s y $L = 100$ mH:

$$C = \frac{1}{L \cdot \omega_c^2} = \frac{1}{0.1 \times (5026.55)^2} = \frac{1}{0.1 \times 25,266,219} = \frac{1}{2,526,622}$$
$$C = 3.958 \times 10^{-7} \text{ F} = 395.8 \text{ nF} \approx 390 \text{ nF} \quad (27)$$

$$R = \sqrt{2}\omega_c \cdot L = 1.41421 \times 5026.55 \times 0.1$$
$$R = 710.8 \Omega \quad (28)$$

2.2.1. Obtención de la función de transferencia $H(s)$

Aplicando la LVK al circuito RLC serie en el dominio de Laplace:

$$V_{in}(s) = \frac{1}{sC} I(s) + R I(s) + sL I(s) = I(s) \left(\frac{1}{sC} + R + sL \right) \quad (29)$$

La salida se toma sobre el inductor:

$$V_{out}(s) = sL \cdot I(s) \quad (30)$$

La función de transferencia es:

$$H(s) = \frac{V_{out}(s)}{V_{in}(s)} = \frac{sL}{\frac{1}{sC} + R + sL} \quad (31)$$

Multiplicando numerador y denominador por sC :

$$H(s) = \frac{s^2 LC}{s^2 LC + sRC + 1} \quad (32)$$



Dividiendo entre LC :

$$H(s) = \frac{s^2}{s^2 + \frac{R}{L}s + \frac{1}{LC}} = \frac{s^2}{s^2 + \sqrt{2}\omega_c s + \omega_c^2} \quad (33)$$

donde $\omega_c = 2\pi(800) = 5026.55$ rad/s. La ganancia es cero en DC ($H(0) = 0$, bloquea la componente continua) y unitaria en alta frecuencia ($\lim_{s \rightarrow \infty} H(s) = 1$), confirmando el comportamiento pasa altas.

2.2.2. Discretización mediante la transformada bilineal (Tustin)

Paso 1: Pre-distorsión de frecuencia.

$$\Omega_a = \tan\left(\frac{\pi f_c}{f_s}\right) = \tan\left(\frac{\pi \cdot 800}{8000}\right) = \tan\left(\frac{\pi}{10}\right) = \tan(18) = 0.32492 \quad (34)$$

$$H(s) = \frac{s^2}{s^2 + \sqrt{2}\Omega_a s + \Omega_a^2} \quad (35)$$

Paso 2: Sustitución bilineal $s = \frac{z-1}{z+1}$.

$$H(z) = \frac{\left(\frac{z-1}{z+1}\right)^2}{\left(\frac{z-1}{z+1}\right)^2 + \sqrt{2}\Omega_a \left(\frac{z-1}{z+1}\right) + \Omega_a^2} \quad (36)$$

Simplificación:

Multiplicando numerador y denominador por $(z+1)^2$:

$$H(z) = \frac{(z-1)^2}{(z-1)^2 + \sqrt{2}\Omega_a (z-1)(z+1) + \Omega_a^2 (z+1)^2} \quad (37)$$

Expandiendo cada término del denominador por separado:

$$(z-1)^2 = z^2 - 2z + 1 \quad (38)$$

$$\sqrt{2}\Omega_a (z-1)(z+1) = \sqrt{2}\Omega_a (z^2 - 1) = 0.45944 z^2 - 0.45944 \quad (39)$$

$$\Omega_a^2 (z+1)^2 = 0.10557 (z^2 + 2z + 1) = 0.10557 z^2 + 0.21114 z + 0.10557 \quad (40)$$



Agrupando el denominador por potencias:

$$D(z) = z^2 \underbrace{(1 + 0.45944 + 0.10557)}_{D_0} + z \underbrace{(-2 + 0 + 0.21114)}_{D_1} + \underbrace{(1 - 0.45944 + 0.10557)}_{D_2} \quad (41)$$

Evaluyendo:

$$D_0 = 1 + \sqrt{2} \Omega_a + \Omega_a^2 = 1 + 0.45944 + 0.10557 = 1.56501 \quad (42)$$

$$D_1 = -2 + 2\Omega_a^2 = -2 + 2(0.10557) = -2 + 0.21114 = -1.78886 \quad (43)$$

$$D_2 = 1 - \sqrt{2} \Omega_a + \Omega_a^2 = 1 - 0.45944 + 0.10557 = 0.64613 \quad (44)$$

Paso 4: Normalizando: Dividiendo numerador y denominador entre $D_0 = 1.56501$:

$$H(z) = \frac{\frac{1}{D_0} (z^2 - 2z + 1)}{z^2 + \frac{D_1}{D_0} z + \frac{D_2}{D_0}} \quad (45)$$

Calculando cada coeficiente:

$$A_0 = \frac{1}{D_0} = \frac{1}{1.56501} = 0.63897 \quad (46)$$

$$A_1 = \frac{-2}{D_0} = \frac{-2}{1.56501} = -1.27795 \quad (47)$$

$$A_2 = \frac{1}{D_0} = \frac{1}{1.56501} = 0.63897 \quad (48)$$

Para los coeficientes del denominador, se utiliza la convención de la ecuación en diferencias $y(k) = \dots + B_1 y(k-1) + B_2 y(k-2)$:

$$B_1 = -\frac{D_1}{D_0} = -\frac{-1.78886}{1.56501} = 1.14305 \quad (49)$$

$$B_2 = -\frac{D_2}{D_0} = -\frac{0.64613}{1.56501} = -0.41286 \quad (50)$$

La función de transferencia discreta resulta:

$$H(z) = \frac{0.63897 z^2 - 1.27795 z + 0.63897}{z^2 - 1.14305 z + 0.41286} \quad (51)$$



2.2.3. Ecuación en diferencias

A partir de $H(z) = Y(z)/U(z)$:

$$Y(z) (z^2 - B_1 z - B_2) = U(z) (A_0 z^2 + A_1 z + A_2) \quad (52)$$

Dividiendo ambos lados entre z^2 y aplicando la transformada z inversa:

$$y(k) - B_1 y(k-1) - B_2 y(k-2) = A_0 u(k) + A_1 u(k-1) + A_2 u(k-2) \quad (53)$$

Reorganizando:

$$\boxed{y(k) = A_0 u(k) + A_1 u(k-1) + A_2 u(k-2) + B_1 y(k-1) + B_2 y(k-2)} \quad (54)$$

Sustituyendo los valores numéricos:

$$y(k) = 0.63897 u(k) - 1.27795 u(k-1) + 0.63897 u(k-2) + 1.14305 y(k-1) - 0.41286 y(k-2) \quad (55)$$

coeficientes:

Tabla 2: Coeficientes del filtro pasa altas de segundo orden ($f_c = 800$ Hz, $f_s = 8000$ Hz).

Coeficiente	Expresión	Valor numérico
A_0	$\frac{1}{D_0}$	0.63897
A_1	$\frac{-2}{D_0}$	-1.27795
A_2	$\frac{1}{D_0}$	0.63897
B_1	$-\frac{D_1}{D_0}$	1.14305
B_2	$-\frac{D_2}{D_0}$	-0.41286

1. **En DC** ($z = 1$):

$$H(1) = \frac{A_0 + A_1 + A_2}{1 - B_1 - B_2} = \frac{0.63897 - 1.27795 + 0.63897}{1 - 1.14305 + 0.41286} = \frac{0}{0.26981} = 0 \quad (56)$$

Esto confirma que el filtro pasa altas bloquea completamente la componente DC.

2. **En Nyquist** ($z = -1$):

$$H(-1) = \frac{A_0 - A_1 + A_2}{1 + B_1 - B_2} = \frac{0.63897 + 1.27795 + 0.63897}{1 + 1.14305 + 0.41286} = \frac{2.55589}{2.55591} \approx 1.0 \quad (57)$$

Esto confirma que la ganancia es unitaria en la frecuencia de Nyquist.

3. **Suma de coeficientes del numerador:**

$$A_0 + A_1 + A_2 = 0.63897 - 1.27795 + 0.63897 = 0 \quad (58)$$

garantiza $H(1) = 0$.

2.2.4. Diagrama de flujo de señal

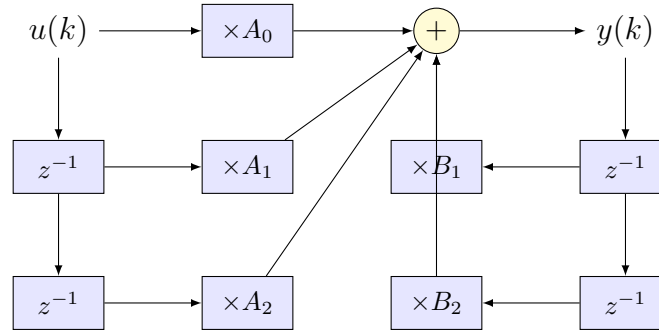


Figura 4: Diagrama de flujo de señal del filtro pasa altas de segundo orden (Forma Directa I).



2.3. Implementación en MicroPython

2.3.1. Documentación del código

El programa comienza importando las bibliotecas necesarias para interactuar con el hardware de la Raspberry Pi Pico 2W. De la biblioteca `machine` se traen las clases `Pin`, `ADC` y `PWM`, que permiten configurar pines digitales, leer señales analógicas y generar ondas con modulación de ancho de pulso, respectivamente. También se importa el módulo `_thread`, que es la herramienta que ofrece MicroPython para trabajar con los dos núcleos del procesador RP2350 de forma simultánea, y el módulo `time`, que se utiliza para medir tiempos y hacer pausas con precisión de microsegundos. Además se importan `sys` y `select`, necesarios para implementar la lectura del puerto serial de forma no bloqueante.

A continuación se definen los parámetros fundamentales del sistema. La frecuencia de muestreo se fija en 8000 Hz, lo que implica que el filtro debe calcular una nueva muestra cada 125 microsegundos. Estos valores se guardan en las constantes `FS` y `PERIOD_US` para que sean fáciles de modificar si en algún momento se quisiera cambiar la tasa de muestreo. También se define la constante `PRINT_SKIP`, que indica cada cuántas muestras se envían datos al puerto serial para su visualización; esto es necesario porque imprimir cada una de las 8000 muestras por segundo desbordaría el buffer USB y provocaría que los comandos dejaran de responder.

Después viene la configuración del hardware. El pin GP26 se configura como entrada analógica a través de la clase `ADC`, ya que es uno de los pines que tiene convertidor analógico-digital integrado en la Pico. El pin GP3 se configura como salida PWM para generar la onda cuadrada que sirve como señal de prueba; se le asigna una frecuencia inicial de 200 Hz y un ciclo de trabajo del 50 %, lo que produce una onda simétrica. Finalmente, el LED integrado de la placa se configura como salida digital para servir como indicador visual de que el sistema se encuentra filtrando activamente.

Los coeficientes de los dos filtros se declaran como constantes globales. Para el filtro pasa bajas de primer orden, que se diseñó a partir del circuito RC con frecuencia de corte de 500 Hz, los coeficientes son $A_0 = A_1 = 0.16591$ y $B_1 = 0.66818$, obtenidos mediante la transformada bilineal como se mostró en la sección anterior. Para el filtro pasa altas de segundo orden, derivado del circuito RLC con frecuencia de corte de 800 Hz, los coeficientes son $A_0 = 0.63897$, $A_1 = -1.27795$, $A_2 = 0.63897$, $B_1 = 1.14305$ y $B_2 = -0.41286$. Estos valores se almacenan con cinco decimales para mantener una precisión adecuada en los cálculos de punto flotante.



El estado compartido entre los dos núcleos se almacena en una lista mutable de Python llamada `st`, que contiene once elementos: dos banderas de control (si el filtro está corriendo y cuál filtro está activo), las variables de estado de ambos filtros (entradas y salidas anteriores), y tres valores para la comunicación con el Núcleo 0 (la última entrada leída, la última salida calculada, y una bandera que indica si hay datos nuevos listos para imprimir). Se eligió una lista mutable porque en el RP2350, el Núcleo 1 no puede acceder de manera confiable a las variables globales del módulo principal; al pasar la lista como argumento a la función del hilo, ambos núcleos pueden leer y escribir los mismos datos sin problema, ya que la lista se pasa por referencia. Para hacer el código más legible, se definen constantes con los nombres de cada índice de la lista.

La parte central del programa es la función `filter_core`, que se ejecuta en el Núcleo 1 del procesador. Esta función recibe como argumentos todo lo que necesita para operar: la lista de estado compartido, el objeto del ADC, el módulo de tiempo, todos los coeficientes de ambos filtros, el periodo de muestreo y el valor de salto de impresión. De esta forma, la función no hace ninguna referencia a nombres globales del módulo, lo que evita el error de visibilidad que tiene el RP2350 con los hilos. Dentro de un ciclo infinito, mientras la bandera de ejecución esté activa, la función registra el tiempo de inicio, lee el valor del ADC convirtiéndolo a voltaje entre 0 y 3.3 V, y evalúa la ecuación en diferencias del filtro seleccionado. Cada cierto número de muestras (determinado por `PRINT_SKIP`), guarda los valores de entrada y salida en la lista compartida y activa la bandera de datos nuevos para que el Núcleo 0 los imprima. Al final de cada iteración, calcula cuánto tiempo tardó el procesamiento y espera el tiempo restante para completar exactamente los 125 microsegundos del periodo de muestreo.

La función `filter_core` se lanza en el Núcleo 1 mediante la instrucción `_thread.start_new_thread`, pasando como argumentos la lista de estado, el objeto del ADC, el módulo de tiempo, los coeficientes de ambos filtros, el periodo y el salto de impresión. De este modo, ambos núcleos del RP2350 trabajan de manera simultánea sin interferirse: el Núcleo 1 se dedica exclusivamente al cálculo del filtro, y nunca accede al puerto serial.

El Núcleo 0 se encarga de toda la comunicación serial. Para poder recibir comandos y al mismo tiempo enviar los datos del filtro al Serial Plotter, se utiliza el mecanismo de `select.poll()`, que permite verificar si hay datos disponibles en la entrada estándar sin quedarse bloqueado esperando. El ciclo principal del Núcleo 0 hace tres cosas en cada iteración: primero revisa si llegó algún carácter por el puerto serial y, si es así, lo agrega a un buffer hasta recibir un



salto de línea, momento en que procesa el comando completo; después revisa si la bandera de datos nuevos está activa en la lista compartida y, si lo está, imprime los valores de entrada y salida en el formato **entrada,salida** que espera el Serial Plotter; y por último hace una pausa breve de 200 microsegundos antes de repetir. Los comandos disponibles son: **START** activa el filtrado y enciende el LED; **STOP** lo detiene, apaga el LED, reinicia las variables de estado del filtro activo y vuelve a mostrar el menú; **LP** selecciona el filtro pasa bajas y reinicia sus variables; **HP** hace lo mismo para el filtro pasa altas; **FREQ** seguido de un número cambia la frecuencia de la onda cuadrada de prueba; y **STATUS** muestra en pantalla qué filtro está seleccionado, si está activo o detenido, y las frecuencias del sistema y de la onda de prueba.

2.3.2. Código fuente

Listing 1: Código MicroPython del sistema de filtros digitales IIR para Raspberry Pi Pico 2W.

```
1  # =====
2  # filtros_iir.py
3  # Sistema de Filtros Digitales IIR - Raspberry Pi Pico 2W
4  #
5  # Filtro 1: Pasa Bajas de 1er Orden (IIR Butterworth, fc=500 Hz)
6  # Filtro 2: Pasa Altas de 2do Orden (Circuito RLC, fc=800 Hz)
7  #
8  # fs = 8000 Hz | T = 125 us
9  # Comunicacion: USB Serial (COM5)
10 #
11 # Arquitectura de doble nucleo:
12 #   Nucleo 0: Comunicacion serial (no bloqueante) e impresion
13 #   Nucleo 1: Ecuacion en diferencias del filtro
14 #
15 # SOLUCION AL BUG DE _thread EN RP2350:
16 # El Core 1 no puede ver nombres del modulo (globals).
17 # TODO lo que necesita el hilo se pasa como argumentos
18 # en _thread.start_new_thread(func, (arg1, arg2, ...)).
19 # =====
20
21 from machine import Pin, ADC, PWM
22 import _thread
```



```
23 import time
24 import sys
25 import select
26
27 # =====
28 # PARAMETROS DEL SISTEMA
29 # =====
30 FS = 8000 # Frecuencia de muestreo (Hz)
31 PERIOD_US = 125 # Periodo de muestreo (us)
32 PRINT_SKIP = 2 # Imprimir cada N muestras (4000 lineas/
    s)
33
34 # =====
35 # CONFIGURACION DE HARDWARE
36 # =====
37 adc = ADC(Pin(26)) # GP26 = ADC0 (entrada analogica)
38 pwm_sq = PWM(Pin(3)) # GP3 = Onda cuadrada de prueba
39 pwm_sq.freq(200) # Frecuencia inicial: 200 Hz
40 pwm_sq.duty_u16(32768) # 50% duty cycle
41 led = Pin("LED", Pin.OUT) # LED indicador
42
43 # =====
44 # COEFICIENTES - FILTRO PASA BAJAS (1er Orden, Butterworth)
45 # Circuito: RC serie (R=3.18k, C=100nF)
46 #  $f_c = 500 \text{ Hz}$  |  $\Omega_a = \tan(\pi \cdot 500 / 8000) = 0.19891$ 
47 #  $H(z) = 0.16591 \cdot (z + 1) / (z - 0.66818)$ 
48 #  $y(k) = A0 \cdot u(k) + A1 \cdot u(k-1) + B1 \cdot y(k-1)$ 
49 # =====
50 LP_A0 = 0.16591
51 LP_A1 = 0.16591
52 LP_B1 = 0.66818
53
54 # =====
55 # COEFICIENTES - FILTRO PASA ALTAS (2do Orden, RLC Butterworth)
56 # Circuito: RLC serie (C=390nF, R=680ohm, L=100mH), salida en L
57 #  $f_c = 800 \text{ Hz}$  |  $\Omega_a = \tan(\pi \cdot 800 / 8000) = 0.32492$ 
```




```
58 #  $H(z) = (0.63897z^2 - 1.27795z + 0.63897) /$   
59 #       $(z^2 - 1.14305z + 0.41286)$   
60 #  $y(k) = A0*u(k) + A1*u(k-1) + A2*u(k-2) + B1*y(k-1) + B2*y(k-2)$   
61 # =====  
62 HP_A0 = 0.63897  
63 HP_A1 = -1.27795  
64 HP_A2 = 0.63897  
65 HP_B1 = 1.14305  
66 HP_B2 = -0.41286  
67  
68 # =====  
69 # ESTADO COMPARTIDO - lista mutable (visible desde ambos cores)  
70 # Se pasa por referencia al hilo como argumento.  
71 #  
72 # st[0] = running      (0 o 1)  
73 # st[1] = active_filter (0=PB, 1=PA)  
74 # st[2] = lp_u1        u(k-1) del pasa bajas  
75 # st[3] = lp_y1        y(k-1) del pasa bajas  
76 # st[4] = hp_u1        u(k-1) del pasa altas  
77 # st[5] = hp_u2        u(k-2) del pasa altas  
78 # st[6] = hp_y1        y(k-1) del pasa altas  
79 # st[7] = hp_y2        y(k-2) del pasa altas  
80 # st[8] = shared_u     (ultimo valor u para imprimir)  
81 # st[9] = shared_y     (ultimo valor y para imprimir)  
82 # st[10] = new_data    (0 o 1)  
83 # =====  
84 st = [0, 0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0]  
85  
86 # Indices con nombre para legibilidad  
87 RUN = 0  
88 FILT = 1  
89 LU1 = 2  
90 LY1 = 3  
91 HU1 = 4  
92 HU2 = 5  
93 HY1 = 6
```



```
94 HY2 = 7
95 SU = 8
96 SY = 9
97 ND = 10
98
99 # =====
100 # NUCLEO 1: ECUACION EN DIFERENCIAS
101 # Recibe TODO por argumento - no usa globals del modulo.
102 # =====
103 def filter_core(st, adc_obj, tm,
104                 lp_a0, lp_a1, lp_b1,
105                 hp_a0, hp_a1, hp_a2, hp_b1, hp_b2,
106                 period, skip):
107     counter = 0
108
109     while True:
110         if st[0] == 0:          # running?
111             tm.sleep_ms(1)
112             continue
113
114         t0 = tm.ticks_us()
115
116         # Lectura del ADC (16 bits: 0-65535)
117         raw = adc_obj.read_u16()
118         u_k = (raw / 65535.0) * 3.3      # Conversion a voltaje (0-3.3
119                                         V)
120
121         if st[1] == 0:          # active_filter == PB
122             # ---- Filtro Pasa Bajas 1er Orden (RC) ----
123             #  $y(k) = A0*u(k) + A1*u(k-1) + B1*y(k-1)$ 
124             y_k = lp_a0 * u_k + lp_a1 * st[2] + lp_b1 * st[3]
125             st[2] = u_k          # lp_u1
126             st[3] = y_k          # lp_y1
127         else:
128             # ---- Filtro Pasa Altas 2do Orden (RLC) ----
129             #  $y(k) = A0*u(k) + A1*u(k-1) + A2*u(k-2)$ 
```



```
129         #          + B1*y(k-1) + B2*y(k-2)
130     y_k = (hp_a0 * u_k + hp_a1 * st[4] + hp_a2 * st[5]
131           + hp_b1 * st[6] + hp_b2 * st[7])
132     st[5] = st[4]      # hp_u2 = hp_u1
133     st[4] = u_k        # hp_u1
134     st[7] = st[6]      # hp_y2 = hp_y1
135     st[6] = y_k        # hp_y1
136
137     # Guardar datos para impresion (cada 'skip' muestras)
138     counter += 1
139     if counter >= skip:
140         st[8] = u_k      # shared_u
141         st[9] = y_k      # shared_y
142         st[10] = 1       # new_data
143         counter = 0
144
145     # Esperar resto del periodo de muestreo
146     elapsed = tm.ticks_diff(tm.ticks_us(), t0)
147     if elapsed < period:
148         tm.sleep_us(period - elapsed)
149
150     # =====
151     # INICIAR NUCLEO 1 - pasar TODO como argumentos
152     # =====
153     _thread.start_new_thread(filter_core, (
154         st, adc, time,
155         LP_A0, LP_A1, LP_B1,
156         HP_A0, HP_A1, HP_A2, HP_B1, HP_B2,
157         PERIOD_US, PRINT_SKIP
158     ))
159
160     # =====
161     # NUCLEO 0: COMUNICACION SERIAL NO BLOQUEANTE + IMPRESION
162     # =====
163     poll_obj = select.poll()
164     poll_obj.register(sys.stdin, select.POLLIN)
```



```
165
166 print("\n" + "=" * 50)
167 print("  Sistema de Filtros Digitales IIR")
168 print("  Raspberry Pi Pico 2W | fs = {} Hz".format(FS))
169 print("=" * 50)
170 print("Comandos disponibles:")
171 print("  START      - Iniciar filtrado")
172 print("  STOP       - Detener filtrado")
173 print("  LP        - Filtro pasa bajas (fc=500 Hz)")
174 print("  HP        - Filtro pasa altas (fc=800 Hz)")
175 print("  FREQ <hz> - Frecuencia de onda cuadrada")
176 print("  STATUS     - Estado actual del sistema")
177 print("=" * 50)
178
179 cmd_buf = ""
180
181 while True:
182     # --- Verificar entrada serial (NO bloqueante) ---
183     if poll_obj.poll(0):
184         ch = sys.stdin.read(1)
185         if ch in ('\n', '\r'):
186             cmd = cmd_buf.strip().upper()
187             cmd_buf = ""
188
189             if not cmd:
190                 pass
191
192             elif cmd == "START":
193                 st[RUN] = 1
194                 led.on()
195                 print("OK: Filtrado iniciado")
196
197             elif cmd == "STOP":
198                 st[RUN] = 0
199                 led.off()
200                 # Reiniciar estados del filtro activo
```



```
201         if st[FILT] == 0:
202             st[LU1] = 0.0
203             st[LY1] = 0.0
204         else:
205             st[HU1] = 0.0
206             st[HU2] = 0.0
207             st[HY1] = 0.0
208             st[HY2] = 0.0
209         print("OK: Filtrado detenido")
210         print("=" * 50)
211         print("Comandos: START STOP LP HP FREQ<hz> STATUS")
212         print("=" * 50)
213
214     elif cmd == "LP":
215         st[RUN] = 0
216         led.off()
217         st[FILT] = 0
218         st[LU1] = 0.0
219         st[LY1] = 0.0
220         print("OK: Pasa Bajas 1er Orden (fc=500 Hz)")
221
222     elif cmd == "HP":
223         st[RUN] = 0
224         led.off()
225         st[FILT] = 1
226         st[HU1] = 0.0
227         st[HU2] = 0.0
228         st[HY1] = 0.0
229         st[HY2] = 0.0
230         print("OK: Pasa Altas 2do Orden (fc=800 Hz)")
231
232     elif cmd.startswith("FREQ"):
233         parts = cmd.split()
234         if len(parts) >= 2:
235             try:
236                 freq = int(parts[1])
```



```
237         pwm_sq.freq(freq)
238         print("OK: Onda cuadrada = {} Hz".format(
239             freq))
240     except ValueError:
241         print("ERR: Frecuencia no valida")
242     else:
243         print("Uso: FREQ <frecuencia_hz>")
244
245     elif cmd == "STATUS":
246         nombre = "Pasa Bajas (fc=500Hz)" if st[FILT] == 0 \
247             else "Pasa Altas (fc=800Hz)"
248         estado = "Activo" if st[RUN] == 1 else "Detenido"
249         print("Filtro: {} | Estado: {} | fs: {} Hz | Onda
250             Cuadrada: {} Hz".format(
251                 nombre, estado, FS, pwm_sq.freq()))
252     else:
253         print("ERR: Comando desconocido: {}".format(cmd))
254
255     elif ch:
256         cmd_buf += ch
257
258     # --- Imprimir datos si hay nuevos ---
259     if st[ND] == 1:
260         st[ND] = 0
261         print("{:.4f},{:.4f}".format(st[SU], st[SY]))
262
263     time.sleep_us(200)
```

2.3.3. Diagrama de conexión física

Para probar el sistema, la señal de prueba generada por el pin GP3 (PWM) se conecta directamente al pin GP26 (ADC0) mediante un cable puente. De esta forma, la onda cuadrada producida por el propio microcontrolador entra al convertidor analógico-digital sin necesidad de componentes externos, lo que permite verificar el funcionamiento de los filtros digitales de

manera sencilla.

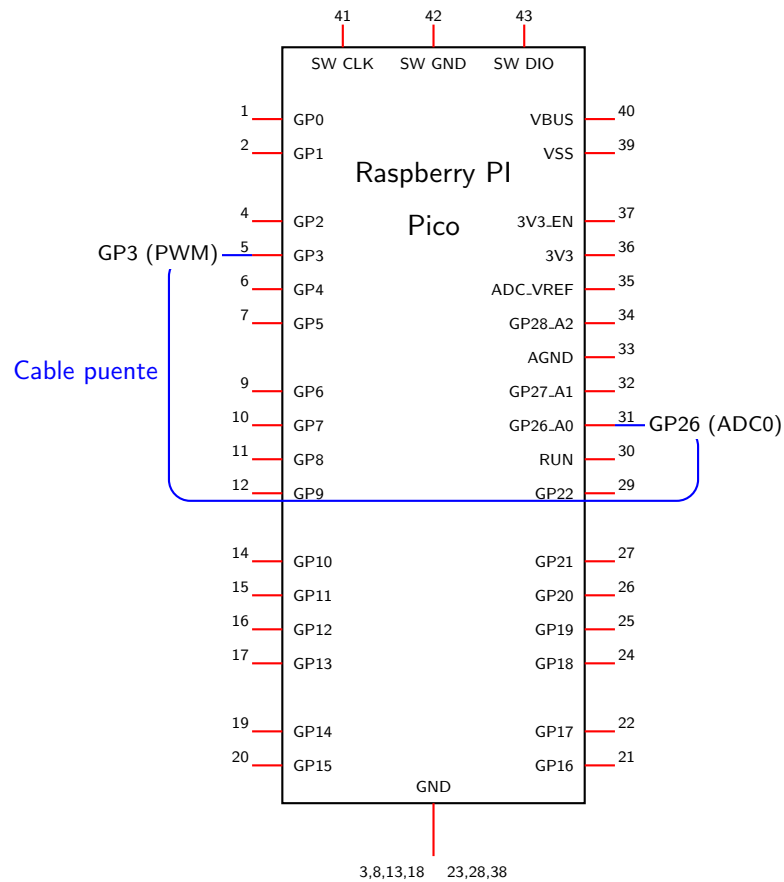


Figura 5: Conexión física: cable puente entre GP3 (salida PWM) y GP26 (entrada ADC) en la Raspberry Pi Pico 2W.



3. Resultados:

3.1. Coeficientes

Tabla 3: Comparación de coeficientes de los filtros diseñados.

Coeficiente	Pasa Bajas 1er Orden	Pasa Altas 2do Orden
A_0	0.16591	0.63897
A_1	0.16591	-1.27795
A_2	0.0	0.63897
B_1	0.66818	1.14305
B_2	0.0	-0.41286
f_c	500 Hz	800 Hz
Tipo	IIR Butterworth	RLC Butterworth
Orden	1	2



3.1.1. Valores esperados con señal de onda cuadrada

Al aplicar una onda cuadrada de frecuencia f_{sq} como señal de prueba, se esperan los siguientes comportamientos:

Tabla 4: Comportamiento esperado con onda cuadrada a distintas frecuencias.

f_{sq} (Hz)	Pasa Bajas ($f_c = 500$ Hz)	Pasa Altas ($f_c = 800$ Hz)
50 Hz	La salida sigue la onda cuadrada con esquinas redondeadas (atenuación mínima).	La salida muestra picos en cada transición, decayendo a cero entre flancos.
200 Hz	Onda cuadrada visible con suavizado moderado de las transiciones.	Picos pronunciados en los flancos; la componente fundamental se atenúa parcialmente.
500 Hz	Señal significativamente suavizada; la fundamental pasa con -3 dB.	Picos visibles; la fundamental pasa parcialmente (-8.9 dB de atenuación).
1000 Hz	Señal muy atenuada y suavizada, salida casi triangular/senoidal.	La onda cuadrada pasa con modificación moderada (-1.5 dB en la fundamental).

3.2. Filtro de primer orden

En esta sección se presentan los resultados obtenidos al aplicar el filtro pasa-bajas de primer orden con frecuencia de corte $f_c = 500$ Hz sobre una señal cuadrada evaluada a distintas frecuencias de entrada: 50, 200, 500 y 1000 Hz.

En cada gráfica se muestra en azul la señal de entrada V_{in} (señal cuadrada) y en naranja la señal de salida V_{out} ya filtrada.

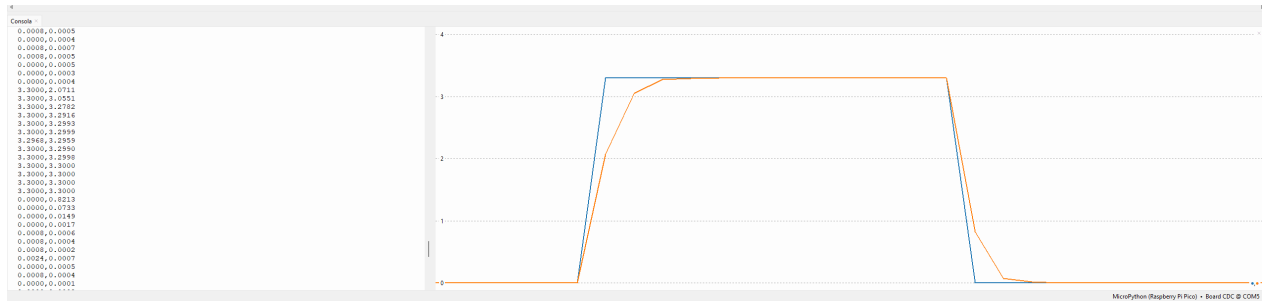


Figura 6: Filtro de primer orden a $f = 50$ Hz.

A 50 Hz, la frecuencia de entrada es muy inferior a la frecuencia de corte $f_c = 500$ Hz, por lo que la señal alterna lentamente y su forma cuadrada es perfectamente distinguible, con flancos abruptos y niveles estables en 3.3 V y 0 V. La señal de salida sigue de cerca a la entrada, presentando únicamente una ligera suavización en los flancos de subida y bajada. La amplitud se conserva prácticamente intacta, lo cual es consistente con que el filtro deja pasar sin atenuación significativa las componentes frecuenciales que se encuentran bien por debajo de f_c .

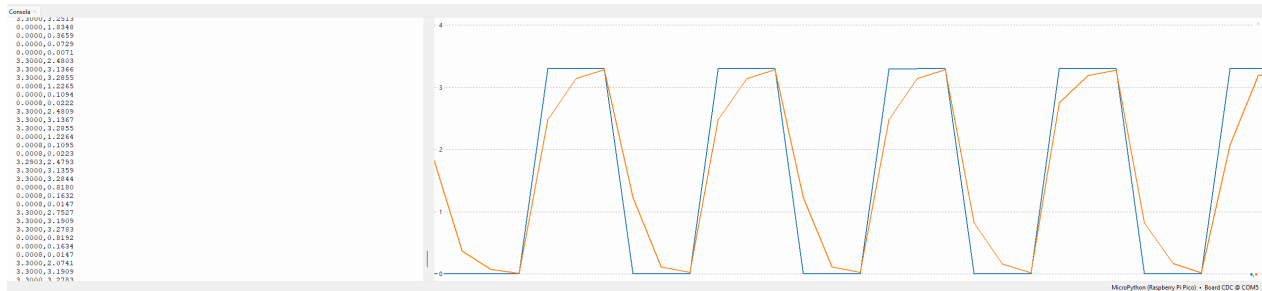


Figura 7: Filtro de primer orden a $f = 200$ Hz.

A 200 Hz la señal de entrada continúa siendo reconocible como cuadrada, aunque los ciclos

son más rápidos. La salida presenta flancos más redondeados: el filtro comienza a atenuar las componentes armónicas de alta frecuencia que conforman los bordes abruptos de la señal cuadrada, resultando en transiciones más graduales. La amplitud de la salida se mantiene cercana a la de la entrada, dado que 200 Hz aún se encuentra en la banda de paso del filtro.

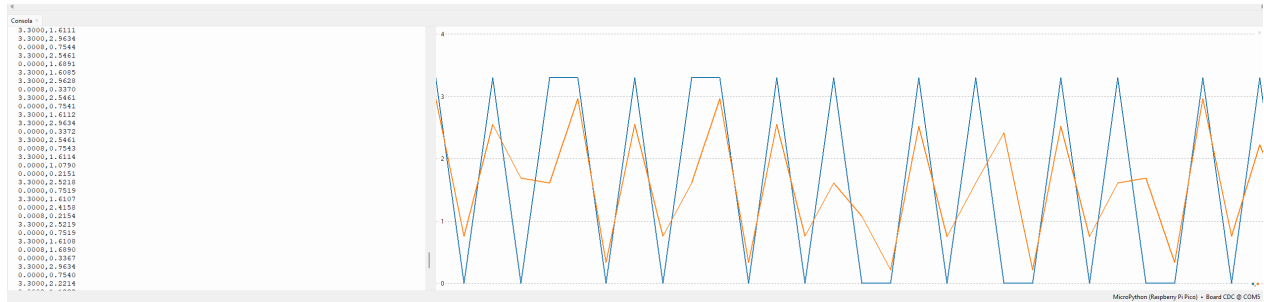


Figura 8: Filtro de primer orden a $f = 500$ Hz.

A 500 Hz, que corresponde exactamente a la frecuencia de corte f_c , la ganancia del filtro es $1/\sqrt{2} \approx 0.707$, equivalente a una atenuación de -3 dB. La señal de salida ha perdido amplitud de forma notable y sus flancos son considerablemente más suaves. La forma cuadrada ya no es distinguible con claridad; en cambio, la señal de salida adopta una apariencia triangular, producto de la atenuación de los armónicos superiores que componen la onda cuadrada.

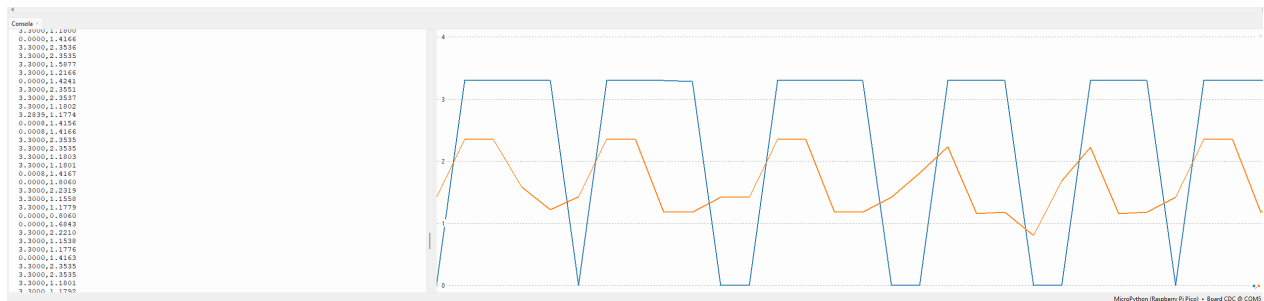


Figura 9: Filtro de primer orden a $f = 1000$ Hz.

A 1000 Hz, el doble de la frecuencia de corte, la atenuación es más pronunciada. La señal de entrada, al alternar con mayor rapidez, se asemeja visualmente a una señal triangular. La señal de salida presenta una amplitud significativamente reducida y sus transiciones están casi completamente suavizadas; el filtro elimina la mayor parte de las componentes armónicas que integran la onda cuadrada, conservando principalmente la componente fundamental.

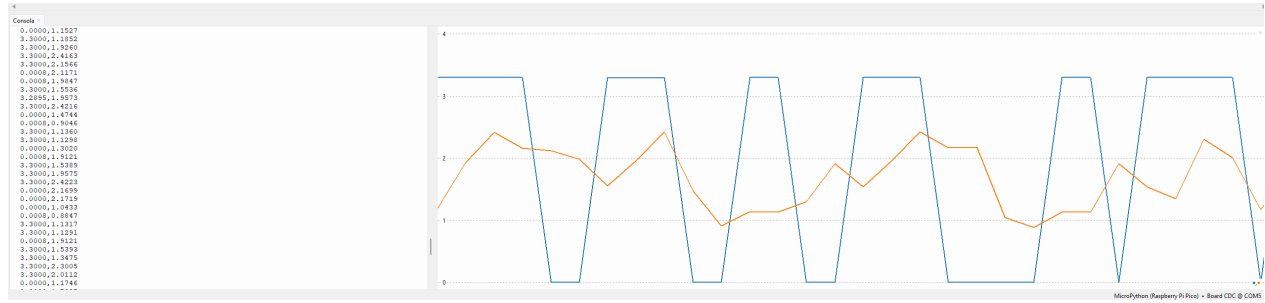


Figura 10: Filtro de primer orden a $f = 2000$ Hz.

3.3. Filtro pasa altas de segundo orden

Ahora se presentan los resultados obtenidos al aplicar el filtro pasa altas de segundo orden (Circuito RLC), con frecuencia de corte $f_c = 800$ Hz, sobre una señal cuadrada evaluada a distintas frecuencias de entrada como lo hicimos para el filtro anterior, 50, 200, 500 y 1000 Hz.

Para comprender el comportamiento observado, es necesario recordar un principio fundamental, y es que la señal de entrada, al ser una onda cuadrada, está compuesta matemáticamente por la suma de múltiples señales senoidales con diferentes frecuencias, es decir, su frecuencia fundamental y una infinidad de armónicos. Es por esto que el filtro reacciona de manera distinta a ciertas partes de la señal, dejando pasar unas componentes frecuenciales y bloqueando otras. Asimismo, se notará en las gráficas que la señal de entrada, a pesar de ser cuadrada, llega a parecerse visualmente a una señal triangular en las frecuencias más altas debido a la enorme rapidez con la que alterna.

Igualmente en cada gráfica se muestra en azul la señal de entrada V_{in} (señal cuadrada) y en naranja la señal de salida V_{out} ya filtrada.

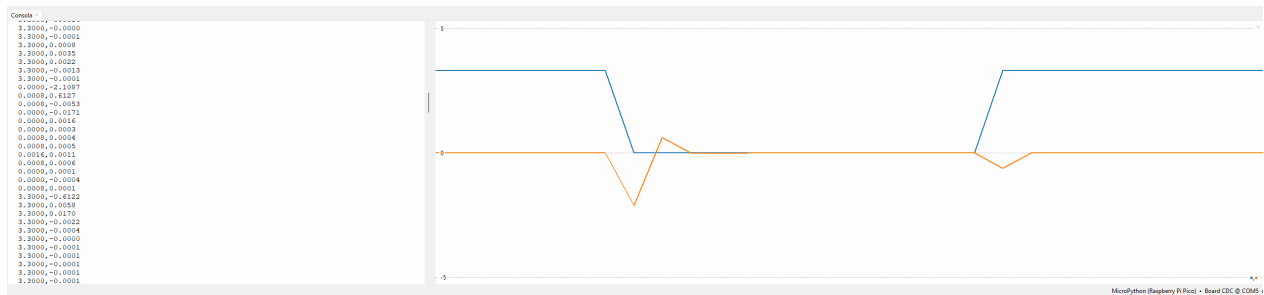


Figura 11: Filtro de segundo orden a $f = 50$ Hz.

A 50 Hz, la frecuencia fundamental de la señal se encuentra muy por debajo de la frecuencia de corte de 800 Hz. En la gráfica se observa que las partes planas de la onda cuadrada, que representan un comportamiento de voltaje constante (frecuencias cercanas a 0 Hz), son bloqueadas casi por completo. Sin embargo, en cada transición abrupta de voltaje (flancos de subida y bajada), la señal de salida presenta picos pronunciados. Esto ocurre porque dichos flancos están compuestos matemáticamente por una suma de armónicos de alta frecuencia, los cuales sí logran atravesar el filtro pasa altas. El resultado es un comportamiento puramente derivador, donde la señal reacciona solo a los cambios y decae a cero en los tramos estables.

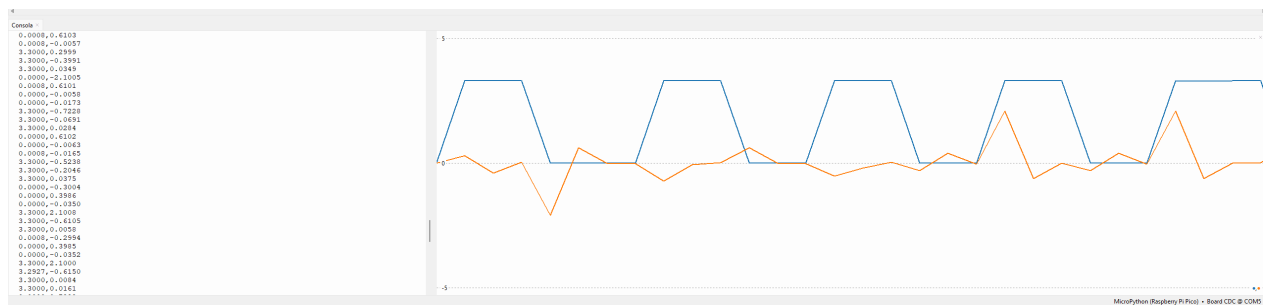


Figura 12: Filtro de segundo orden a $f = 200$ Hz.

A 200 Hz, la señal continúa en la banda de rechazo del filtro. Los picos generados por la alta frecuencia de los flancos siguen siendo la característica dominante de la señal de salida. No obstante, debido a que el periodo de la onda cuadrada es más corto, la respuesta transitoria de la señal filtrada no tiene el tiempo suficiente para decaer completamente a cero antes de que ocurra el siguiente flanco. A pesar de esto, la componente fundamental de 200 Hz sigue estando severamente atenuada.

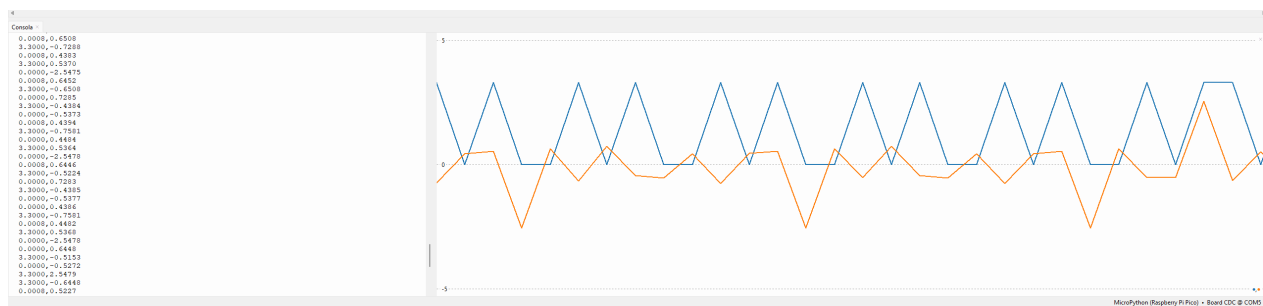


Figura 13: Filtro de segundo orden a $f = 500$ Hz.

A 500 Hz, nos aproximamos a la frecuencia de corte del filtro. Los picos correspondientes a los flancos se ensanchan visiblemente en la base, lo que indica que la componente fundamental comienza a pasar de manera parcial. De acuerdo con el análisis teórico, a esta frecuencia se presenta una atenuación de -8.9 dB. La señal de salida empieza a sostener la amplitud de forma más estable entre las transiciones, intentando replicar la forma de la onda cuadrada original, aunque con evidente distorsión.

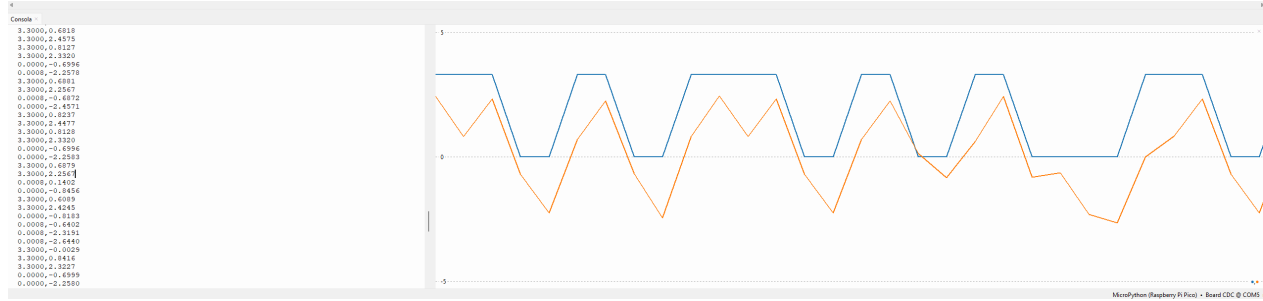


Figura 14: Filtro de segundo orden a $f = 1000$ Hz.

A 1000 Hz, la frecuencia fundamental de la señal ha superado la frecuencia de corte ($f_c = 800$ Hz), ingresando por completo a la banda de paso. La atenuación teórica de la fundamental se reduce drásticamente a solo -1.5 dB. Visualmente, los picos pronunciados y el efecto derivador en los flancos desaparecen. La señal de salida (naranja) ahora es capaz de seguir fielmente los niveles altos y bajos de la señal de entrada (azul), demostrando que la mayor parte de la energía frecuencial de la onda logra atravesar el circuito digital.



3.4. Diferencias entre el Filtro de Primer Orden y Segundo Orden

Entre mayor es el orden de un filtro, mayor es la atenuación que ejerce sobre la señal; por ejemplo, en los filtros de Primer Orden la transición es más suave y lenta, mientras que en un filtro de Segundo Orden la transición es más breve y abrupta.

Aunque en esta práctica el filtro de Primer Orden es un pasa bajas y el de Segundo Orden es un pasa altas, podemos apreciar con claridad este comportamiento al observar la respuesta del sistema, donde el de segundo orden demuestra una mayor selectividad para rechazar las frecuencias no deseadas. Este comportamiento también se ve reflejado en el dominio digital al obtener la ecuación en diferencias. En un filtro de primer orden, la salida depende únicamente de una muestra anterior de la entrada ($u(k-1)$) y una de la salida ($y(k-1)$), de forma que para realizar los cálculos el sistema solo toma en cuenta un valor atrás ya procesado. En cambio, el filtro de segundo orden requiere el doble de información histórica ($k-1$ y $k-2$), lo que lo vuelve más complejo.



4. Conclusiones:

■ Espinoza Matamoros Percival Ulises:

A partir de la realización de esta práctica pude retomar conceptos vistos en asignaturas anteriores, de forma que a partir de diseñar un filtro y a partir de esta obtener sus funciones de transferencia, al tener que implementar el filtro en la Raspberry Pi Pico, pude aprender y comprender la importancia así como la necesidad de discretizar la función de transferencia, para así poder implementarla en un microcontrolador, donde por medio de una señal cuadrada de entrada se pudo observar el comportamiento del filtro, comprobando así que el filtro se comporta de la forma esperada, es decir, que se atenúan las frecuencias bajas para el caso del filtro pasa altas, y se atenúan las frecuencias altas para el caso del filtro pasa bajas. Por otro lado al implementar estos dos filtros se pudo observar que el filtro de primer orden tiene una respuesta más lenta, es decir, que tarda más en estabilizarse, mientras que el filtro de segundo orden tiene una respuesta más rápida, es decir, que tarda menos en estabilizarse. De forma que en la raspberry pi pico se pueden implementar filtros digitales de forma sencilla, y con un buen desempeño, siempre y cuando se tenga una buena discretización de la función de transferencia y se realicen los cálculos correspondientes de forma correcta.

- **Flores Colin Victor Jaziel:** Gracias a esta práctica, pude comprobar cómo el orden de un filtro puede definir de manera crítica el comportamiento de la señal de salida y es que quedó evidenciado que, a mayor orden, la atenuación ejercida es superior: mientras el filtro de primer orden presenta una transición más suave y lenta, el de segundo orden ofrece una respuesta mucho más breve y abrupta y este fenómeno se manifestó claramente al observar la respuesta del sistema, a pesar de que en nuestro montaje experimental el filtro de primer orden se configuró como pasa bajas y el de segundo orden como pasa altas.

La implementación en MicroPython fue de suma importancia para facilitar el análisis de cómo el orden impacta directamente en la complejidad del algoritmo y cuando obtuvimos las ecuaciones en diferencias, observé que un filtro de primer orden es computacionalmente ligero, pues solo requiere sólo un valor atrás tanto de la entrada como de la salida ($u(k-1)$ y $y(k-1)$). En contraste, el de segundo orden demanda el doble de información histórica y operaciones matemáticas adicionales. Esto me permitió concluir que el diseño de filtros digitales depende de las necesidades que hay que resolver y los



recursos de procesamiento disponibles en el microcontrolador.

■ **García Cortés Adolfo de Jesús:**

Esta práctica ayudó a repasar lo aprendido en clases pasadas sobre señales y a aplicarlo en un dispositivo real. Al diseñar los filtros RC, quedó claro que para pasarlos de la teoría analógica a la Raspberry Pi Pico, era obligatorio convertirlos a formato digital mediante matemáticas (usando la transformada bilineal). De esta forma se obtuvo la ecuación necesaria para programarlos.

Probar los filtros con una señal cuadrada fue muy útil porque, por su naturaleza, este tipo de señal está formada por la suma de muchas frecuencias distintas. Esto dejó ver claramente qué hace cada filtro: el pasa-bajas dejó pasar las señales lentas y frenó las rápidas, mientras que el pasa-altas hizo exactamente lo contrario.

Al comparar los filtros, se notó una diferencia importante: el filtro de primer orden bloquea las frecuencias no deseadas de una forma suave, pero el de segundo orden las corta de manera mucho más brusca. Esto también afecta la programación, ya que el de primer orden es más sencillo al solo necesitar el valor anterior de la entrada y la salida ($u(k-1)$ y $y(k-1)$). En cambio, el de segundo orden necesita los dos valores anteriores, lo que le exige más cálculos a la placa.

Por último, quedó demostrado que la Raspberry Pi Pico funciona muy bien para crear filtros digitales, siempre y cuando las conversiones matemáticas y la velocidad para tomar los datos sean las correctas. Los resultados físicos fueron iguales a lo que decía la teoría, lo que confirma que tanto el diseño como el código en MicroPython se hicieron bien.

- **Lara Hernandez Angel Husiel:** La realización de esta práctica me permitió integrar los conceptos teóricos de los filtros analógicos RC con las herramientas de procesamiento digital de señales. Al implementar los algoritmos en la Raspberry Pi Pico, comprendí que la transformación de una función de transferencia del dominio de Laplace al dominio discreto mediante la transformada bilineal es un paso fundamental para que el microcontrolador pueda interpretar el comportamiento físico deseado y esto lo puede apreciar de mejor manera durante las pruebas con la señal cuadrada donde note cómo cada filtro extrae componentes distintas de la señal: el suavizado de las transiciones en el pasa bajas y el aislamiento de los flancos en el pasa altas.



Después de obtener las gráficas y hacer la comparación entre el primer y segundo orden pude visualizar la atenuación que ofrece cada tipo de filtro. Pude constatar que, aunque el filtro de segundo orden es más eficiente para aislar bandas de frecuencia específicas gracias a su corte más drástico, requiere una programación más cuidadosa de las ecuaciones en diferencias para asegurar la estabilidad del sistema. En esta práctica aprendí que un buen diseño matemático con una buena correcta implementación en hardware permite emular componentes analógicos con gran precisión, aprovechando la flexibilidad que ofrecen herramientas como Thony de MicroPython.

■ **Lugo Manzano Rodrigo:**

A lo largo de esta práctica, pude recordar lo ya aprendido en algunas asignaturas pasadas y ver cómo la teoría de circuitos analógicos se traslada al mundo digital mediante herramientas matemáticas. Pude observar cómo la señal cuadrada de entrada responde a los filtros dependiendo de su frecuencia, notando claramente cómo el pasa bajas suaviza la onda al eliminar los armónicos rápidos, mientras que el pasa altas aísla los cambios bruscos de los flancos. También fue evidente que subir a un segundo orden resulta en un corte de frecuencias mucho más estricto frente a la transición suave del primer orden, sin embargo, un reto y aprendizaje importante también estuvo en la implementación física. Trasladar las ecuaciones en diferencias al microcontrolador requirió de una buena optimización para no perder el ritmo de la frecuencia de muestreo establecida. Para lograrlo, fue importante aprovechar la arquitectura de doble núcleo de la Raspberry Pi Pico 2W. Al aislar la ejecución matemática en el Núcleo 1, garantizamos que el filtro calculara cada muestra exactamente cada $125\mu s$, mientras que el Núcleo 0 se encargaba de la comunicación serial de forma no bloqueante. Esto me demostró que en el diseño no basta con tener un modelo matemático perfecto, sino que la eficiencia del código, la sincronización de los núcleos y el manejo adecuado de los recursos del hardware igual son partes importantes para lograr un procesamiento de señales digitales estable.



Referencias

- Butterworth, S. (1930). On the Theory of Filter Amplifiers. *Wireless Engineer*, 7(6), 536-541.
- MicroPython Contributors. (2024a). *_thread — Multithreading Support*. https://docs.micropython.org/en/latest/library/_thread.html
- MicroPython Contributors. (2024b). *class ADC — Analog to Digital Conversion*. <https://docs.micropython.org/en/latest/library/machine.ADC.html>
- MicroPython Contributors. (2024c). *class PWM — Pulse Width Modulation*. <https://docs.micropython.org/en/latest/library/machine.PWM.html>
- MicroPython Contributors. (2024d). *machine — Functions Related to the Hardware*. <https://docs.micropython.org/en/latest/library/machine.html>
- MicroPython Contributors. (2024e). *time — Time Related Functions*. <https://docs.micropython.org/en/latest/library/time.html>
- Ogata, K. (2010). *Modern Control Engineering* (5.^a ed.). Pearson.
- Python Software Foundation. (2024). *_thread — Low-Level Threading API*. https://docs.python.org/3/library/_thread.html
- Raspberry Pi Ltd. (2024a). *Raspberry Pi Pico 2 W Datasheet*. <https://datasheets.raspberrypi.com/pico/pico-2-w-datasheet.pdf>
- Raspberry Pi Ltd. (2024b). *RP2350 Datasheet: A Microcontroller by Raspberry Pi*. <https://datasheets.raspberrypi.com/rp2350/rp2350-datasheet.pdf>
- Sedra, A. S., Smith, K. C., Carusone, T. C., & Gaudet, V. (2020). *Microelectronic Circuits* (8.^a ed.). Oxford University Press.